

## Overview



Football is the world's most popular sport, with 3.4 billion fans [Sport For Business, 2024]. Its quadrennial flagship event, the World Cup, is contested among national teams and is the pinnacle of international recognition for the sport.

'Messi', 'Ronaldo'. Even if you have never watched a football game, the chances that you know either one of those names is relatively high. Those two men have transcended the sport and sparked unending debates for over two decades. However, for all their incredible achievements, none has managed to hoist the World Cup atop his head.

It has been clear for a long time that the debate will remain unending, but what is clear to everyone is that whoever secures the World Cup will edge the G.O.A.T (Greatest Of All Time) shouts.



The 2022 World Cup, scheduled for November-December in Qatar, breaks from Tradition due to weather conditions. This unique timing, coupled with the fact that football legends Ronaldo and Messi are in their late thirties, adds a layer of significance to this event. It could be their last chance to claim the coveted World Cup title.

Undertaking a seemingly daunting task, I set out to predict the 2022 World Cup winner. While sportsbooks do this routinely, I soon realised it was no easy feat. The complexity of the task became apparent as I delved into the data and the intricacies of the game.

## **DATA WRANGLING**

### **Identifying the Data**

My data collection process was meticulous. I gathered the necessary data from FIFA's website, which holds a comprehensive record of previous editions. To ensure accuracy, I cross-referenced this with publicly available data on Wikipedia, a reliable source for historical sports data.

Amazingly, every game played from the first edition in 1930 to the last in 2018 was publicly available. I checked the underlying HTML code for a couple of them, and they were identical - the data could be scraped! At this point, I could see that this project was coming together and wasn't longer an idea.

## Scraping the Data

My tool for extracting the data was the Jupyter Notebook, which runs on the Python programming language. I set it up on my local network and began writing the code.

The first challenge was to gather the fixtures for the 2022 World Cup, which would form the basis of the prediction model. This involved scraping the data from reliable sources and ensuring its accuracy.

```
1  import pandas as pd
2  from string import ascii_uppercase as groups
3  import pickle
4
5  # Extracting all the groups from the website
6  GroupTable = pd.read_html('https://web.archive.org/web/20221115040351/
7  GroupTable[12] # The first group is the 12th table by index search
8  GroupTable[19] # Each of the tables are seperated by 7 other tables
9  GroupTable[26]
```

The tournament begins with a group stage, where 32 teams are divided into 8 groups. Four teams per group play a match against each other, and the top two teams move on to the next round. I extracted one of the 8 group standings from the Wikipedia page. There were several tables on the website, so I used indexing to find the required table.

|   | Pos | Teamvte      | Pld | W | D | L | GF | GA | GD | Pts | Qualification             |
|---|-----|--------------|-----|---|---|---|----|----|----|-----|---------------------------|
| 0 | 1   | Argentina    | 0   | 0 | 0 | 0 | 0  | 0  | 0  | 0   | Advance to knockout stage |
| 1 | 2   | Saudi Arabia | 0   | 0 | 0 | 0 | 0  | 0  | 0  | 0   | Advance to knockout stage |
| 2 | 3   | Mexico       | 0   | 0 | 0 | 0 | 0  | 0  | 0  | 0   | NaN                       |
| 3 | 4   | Poland       | 0   | 0 | 0 | 0 | 0  | 0  | 0  | 0   | NaN                       |

I then wrote a for loop to extract all the groups and place them within a dictionary, removing the 'Qualification' column, which was surplus to requirements.

```

10
11 # Renaming each extracted group with appropriate group name
12 dict_table = {}
13 for x, i in zip(groups, range(12, 64, 7)):
14     df = GroupTable[i]
15     df.rename(columns = {df.columns[1] : 'Team'}, inplace = True),
16     df.drop(columns = df.columns[-1], inplace = True)
17     dict_table[f'Group {x}'] = df
18
19
20 dict_table.keys() # Checking for errors
21 dict_table['Group E']
22
23 with open('groupstage', 'wb') as file:
24     pickle.dump(dict_table, file) # Exporting the dictionary
25

```

I exported the dictionary after checking for errors.

## Organising the Data

I created a data frame containing all the previous World Cup editions (after several hours of extraction, I discovered that the 1990 page HTML was different from the others and brought in incomplete data). I used the BeautifulSoup package to scrape the data, extracting every match played within a single edition of the World Cup.

```

27  import requests
28  from bs4 import BeautifulSoup
29  from pandas.api.types import CategoricalDtype
30
31  years1 = [1930, 1934, 1938, 1950, 1954, 1958, 1962,
32           |      |      |      |      |      |      |      |
33           1966, 1970, 1974, 1978, 1982, 1986]
34
35  years2 = [1994, 1998, 2002, 2006, 2010, 2014, 2018]
36
37  url = 'https://en.wikipedia.org/wiki/2018_FIFA_World_Cup'
38  response = requests.get(url)
39  file = response.text
40
41  soup = BeautifulSoup(file, 'lxml')
42
43  matches = soup.find_all('div', class_="footballbox")

```

Next, I created empty lists to collect the extracted data based on the home team name, scoreline, and away team name. For each list, I created a for loop that iterated through all the games played in a single edition and appended the scraped data to its respective list. I saved the lists to a dictionary and then created a data frame from the dictionary.

```

44 # Creating empty lists for each game played
45 home = []
46 score = []
47 away = []
48
49 # Appending the scraped information from the website into the empty lists
50 for match in matches:
51     home.append(match.find('th', 'fhome').get_text())
52     score.append(match.find('th', 'fscore').get_text())
53     away.append(match.find('th', 'faway').get_text())
54
55 dict_matches = {'home':home, 'score':score, 'away':away}
56
57 # Saving the dictionary into a dataframe
58 df_games = pd.DataFrame(dict_matches)
59 df_games['year'] = '2018'
60 df_games

```

Simply put, I scraped all the games played at the 2018 World Cup (the latest edition), extracted the team names and score lines and saved them into a data frame.

```
[54]:
```

|     | home    | score        | away         | year |
|-----|---------|--------------|--------------|------|
| 0   | Russia  | 5-0          | Saudi Arabia | 2018 |
| 1   | Egypt   | 0-1          | Uruguay      | 2018 |
| 2   | Russia  | 3-1          | Egypt        | 2018 |
| 3   | Uruguay | 1-0          | Saudi Arabia | 2018 |
| 4   | Uruguay | 3-0          | Russia       | 2018 |
| ... | ...     | ...          | ...          | ...  |
| 59  | Russia  | 2-2 (a.e.t.) | Croatia      | 2018 |
| 60  | France  | 1-0          | Belgium      | 2018 |
| 61  | Croatia | 2-1 (a.e.t.) | England      | 2018 |
| 62  | Belgium | 2-0          | England      | 2018 |
| 63  | France  | 4-2          | Croatia      | 2018 |

64 rows × 4 columns

Seeing that the scraping was successful and error-free, I created a function that would iterate through every World Cup edition (2018, 2014, 2010, etc.), scrape the data, and save it into a data frame. Inside the function, I added a loop that would create a column and add the respective years to identify the games per year easily.

```

62 # Creating a function to use for all previous editions
63 def all_games_played(year):
64     url = f'https://en.wikipedia.org/wiki/{year}_FIFA_World_Cup'
65     response = requests.get(url)
66     file = response.text
67
68     soup = BeautifulSoup(file, 'lxml')
69
70     matches = soup.find_all('div', class_="footballbox")
71
72     home = []
73     score = []
74     away = []
75
76     for match in matches:
77         home.append(match.find('th', 'fhome').get_text())
78         score.append(match.find('th', 'fscore').get_text())
79         away.append(match.find('th', 'faway').get_text())
80
81     dict_matches = {'home':home, 'score':score, 'away':away}
82
83     df_games = pd.DataFrame(dict_matches)
84     df_games['year'] = year
85
86     return df_games
87
88 all_games_played('1990')
89

```

While revising the extracted data, I noticed a problem with the 1990 World Cup data. Only the knockout stage matches conformed to the scraping code. I had thirty-six matches missing (the group stages).

I had to think of the most efficient way to combine these thirty-six matches with the rest of the correctly extracted matches. After much googling and stack overflow, I manually entered the matches (not the most efficient way, but they seemed small enough to get done quickly).

```

90 # Creating a dataframe for the missing data in 1990
91 df_missing_data = pd.DataFrame(
92     {'home': ['Italy', 'United States', 'Italy', 'Austria',
93             'Argentina', 'Soviet Union', 'Argentina', 'C
94             'Brazil', 'Costa Rica', 'Brazil', 'Sweden', '
95             'United Arab Emirates', 'West Germany', 'Yugo
96             'Belgium', 'Uruguay', 'Belgium', 'South Korea
97             'England', 'Netherlands', 'England', 'Republi
98             'score': ['1-0', '1-5', '1-0', '0-1', '2-0', '2-1',
99                     '0-1', '1-0', '2-0', '2-1', '1-1', '0-4',
00                     '2-1', '1-0', '1-0', '1-2', '1-0', '1-2',
01                     '0-2', '4-1', '1-0', '5-1', '1-1', '4-1',
02                     '2-0', '0-0', '3-1', '1-3', '1-2', '0-1',
03                     '1-1', '1-1', '0-0', '0-0', '1-0', '1-1'],
04             'away': ['Austria', 'Czechoslovakia', 'United States'
05                     'Cameroon', 'Romania', 'Soviet Union', 'Roman
06                     'Sweden', 'Scotland', 'Costa Rica', 'Scotland
07                     'Colombia', 'Yugoslavia', 'Colombia', 'United
08                     'South Korea', 'Spain', 'Uruguay', 'Spain',
09                     'Republic of Ireland', 'Egypt', 'Netherlands'
10
11 df_missing_data['year'] = '1990' # Adding the year column
12 df_missing_data

```

I had all the data I needed within different data frames, and the next step was to combine everything into a single data frame, which would be fed into the model.

I combined the missing 1990 data with the scraped knockout data.

I combined all the matches played before and after 1990.

I combined the matches played before 1990, the 1990 data, and those played after 1990 into a single data frame and saved it as a CSV.

```

111 df_missing_data['year'] = '1990'          # Adding the year column
112 df_missing_data
113
114 # Creating a dataframe for the extracted 1990 data
115 df_1990_2 = all_games_played('1990')
116
117 # Appending the extracted data to the missing data
118 df_1990 = df_missing_data._append(df_1990_2, ignore_index = True)
119 df_1990
120
121 # For loops using list comprehension
122 before_1990 = [all_games_played(year) for year in years1]
123 after_1990 = [all_games_played(year) for year in years2]
124
125 # Extracting all world cup games data excluding 1990
126 df_before_1990 = pd.concat(before_1990, ignore_index=True)
127 df_after_1990 = pd.concat(after_1990, ignore_index = True)
128
129 # Adding 1990 data within the extracted data
130 df_world1 = df_before_1990._append(df_1990, ignore_index=True)
131 df_world_cup = df_world1._append(df_after_1990, ignore_index=True)
132 df_world_cup
133
134 # Saving the dataframe to a csv
135 df_world_cup.to_csv('World_cup_total_games_2018.csv', index=False)

```

At this point, I had the tournament's group stage format and the data from all previous editions from which the model would learn, but something important was still missing—the fixtures. I needed to scrape the schedule for how all the matches would be played, from the group stages to the final.

I used the BeautifulSoup package within a for loop to extract this and saved it into a CSV.

```

137 # Extracting the fixtures for the 2022 world cup
138 url2 = 'https://web.archive.org/web/20221115040351/https://en.wikiped
139 response = requests.get(url2)
140 file2 = response.text
141
142 soup2 = BeautifulSoup(file2, 'lxml')
143
144 matches = soup2.find_all('div', class_="footballbox")
145
146 home = []
147 score = []
148 away = []
149
150 for match in matches:
151     home.append(match.find('th', 'fhome').get_text())
152     score.append(match.find('th', 'fscore').get_text())
153     away.append(match.find('th', 'faway').get_text())
154
155 dict_matches = {'home':home, 'score':score, 'away':away}
156
157 df_fixtures = pd.DataFrame(dict_matches)
158 df_fixtures['year'] = 2022
159 df_fixtures
160
161 # Saving the dataframe to a csv
162 df_fixtures.to_csv('World_cup_fixtures_2022.csv', index=False)

```

## Cleaning the Data

I had the data I needed, but it still needed cleaning. I assigned the CSVs to new data frames and began cleaning (removing whitespaces, assigning appropriate names and datatypes, removing unnecessary information, etc.).

```

171 # Removing all leading and trailing whitespaces
172 df_fifa['home'] = df_fifa['home'].str.strip()
173 df_fifa['away'] = df_fifa['away'].str.strip()
174 df_fixture['home'] = df_fixture['home'].str.strip()
175 df_fixture['away'] = df_fixture['away'].str.strip()
176
177 df_fifa
178
179 # There is a game in which there was a walkover that needs to be re
180
181 to_delete = df_fifa[df_fifa['score'].str.contains('w/o')].index #
182 df_fifa.drop(index = to_delete, inplace= True)
183
184 # Removing all scores that have (a.e.t)
185 df_fifa['score'] = df_fifa['score'].str.replace('[(a-z)+.]', '', reg
186
187 # Removing leading or trailing whitespaces
188 df_fifa['score'] = df_fifa['score'].str.strip()
189
190 # Assigning the home and away goals respectively
191 df_fifa[['home_goals', 'away_goals', 'x']] = df_fifa['score'].str.spl
192
193 # Dropping the unnecessary columns
194 df_fifa.drop(['score', 'x'], axis=1, inplace=True)
195
196 # Checking the datatypes
197 df_fifa.dtypes
198
199 # Assingning appropriate datatypes
200 df_fifa = df_fifa.astype({'home_goals':int, 'away_goals':int})

```

\*I ran into an issue when trying to separate the score columns. For some reason, it refused to split appropriately. I imported the CSV into MS Excel, split it there and exported it back to the Jupyter Notebook\*

I did a final check for all the data extracted and then saved it to another CSV.

```

202 # Final check to see if all data was entered properly
203 years3 = [1930, 1934, 1938, 1950, 1954, 1958, 1962, 1966, 1970, 1974
204 |         | 1978, 1982, 1986, 1994, 1998, 2002, 2006, 2010, 2014, 2018
205 for year in years3:
206 |     print(year, len(df_fifa[df_fifa['year']==year]))
207
208 # Saving the files
209 df_fifa.to_csv('World_cup_data_cleaned.csv', index=False)
210 df_fixture.to_csv('World_cup_fixtures_2022_cleaned.csv', index=False)

```

## Building the Prediction Model

I had the group stage format, historical data and 2022 World Cup fixtures, and the next step was to feed in the data and build the prediction model.

I created data frames for home and away teams based on goals scored and goals conceded for every match played from the first edition to the last. From the resulting data frame, I calculated the average goals scored per team. I did this to reward teams that have consistently scored more over the period while training the model.

```

212 from scipy.stats import poisson
213
214 group_stage = pickle.load(open('groupstage', 'rb'))
215 df_world = pd.read_csv('World_cup_data_cleaned.csv')
216 df_plays = pd.read_csv('World_cup_fixtures_2022_cleaned.csv')
217
218 group_stage['Group A']
219
220 df_home = df_world[['home', 'home_goals', 'away_goals']]
221 df_away = df_world[['away', 'home_goals', 'away_goals']]
222 df_home = df_home.rename(columns = {'home': 'Team', 'home_goals': 'Goals'})
223 df_away = df_away.rename(columns = {'away': 'Team', 'home_goals': 'Goals'})
224
225 df_away
226
227 # Calculating the average goals scored by team over the previous editions
228 df_avg_goals = pd.concat([df_home, df_away]).groupby('Team').mean()
229 df_avg_goals

```

Next, I had to create the model.

This part took a lot of tweaks, review, YouTube and googling, but I finally got it done! I used the Poisson distribution as the event assumptions were correlating;

- The events could be counted (matches to be played)
- The occurrence of events is independent (Scoring a goal doesn't directly affect the other team's ability to score as well)
- The rate at which events occur is constant (every match is 90 minutes)
- No two goals can be scored at the same time in a match

```
233 def prediction(home, away):
234     if home in df_avg_goals.index and away in df_avg_goals.index:
235         lambda_home = df_avg_goals.at[home, 'Goals_scored'] * df_avg
236         lambda_away = df_avg_goals.at[away, 'Goals_scored'] * df_avg
237         prob_home_win, prob_away_win, prob_draw = 0, 0, 0
238         for x in range(0,11):
239             for y in range(0,11):
240                 p = poisson.pmf(x, lambda_home) * poisson.pmf(y, la
241                 if x==y:
242                     prob_draw += p
243                 elif x > y:
244                     prob_home_win += p
245                 else:
246                     prob_away_win += p
247             home_points = 3 * prob_home_win + prob_draw
248             away_points = 3 * prob_away_win + prob_draw
249             return (home_points, away_points)
250         else:
251             return (0, 0)
252
253 # Testing out the function
254 prediction('Argentina', 'Uruguay')
```

The Poisson package did most of the heavy lifting. I multiplied the goals scored and conceded for each home and away team, set 10 as the maximum number of goals that could be scored, and then inserted the values for x, y, and lambda. All variables were to begin from initial states of zero.

Lastly, the function would reward teams 3 points for a win and 1 point for a draw.

I tested the function, and it was working as designed. The final step in building the model was to structure the tournament as it was scheduled to run. I split the fixtures based on the group stages and knockout rounds.

```
256 # Structuring the games based on level
257 df_group_stage = df_plays[:48].copy() # Group stage games
258 df_R_16 = df_plays[48:56].copy() # The round of 16 teams
259 df_Qf = df_plays[56:60].copy() # The Quarter final games
260 df_Sf = df_plays[60:63].copy() # The semi-final and 3rd place
261 df_F = df_plays[63:].copy() # The final game
262
263 df_group_stage
```

## The Group Stage

Firstly, I extracted all team names and assigned them to a data frame, the first team being home and the second being away; I used the function to predict a game between such two teams, find the teams in the group stage format, assign points based on the result, and then reveal the points accumulated for each team. I wrapped this all up in a for loop so it iterates through every group and shows the final group standings.

```
265 for group in group_stage:
266     team_names = group_stage[group]['Team'].values
267     df_group_games = df_group_stage[df_group_stage['home'].isin(team_names)]
268     for index, row in df_group_games.iterrows():
269         home, away = row['home'], row['away']
270         home_points, away_points = prediction(home, away)
271         group_stage[group].loc[group_stage[group]['Team'] == home, 'Pts'] += home_points
272         group_stage[group].loc[group_stage[group]['Team'] == away, 'Pts'] += away_points
273
274     group_stage[group] = group_stage[group].sort_values('Pts', ascending=False)
275     group_stage[group] = group_stage[group][['Team', 'Pts']]
276     group_stage[group] = group_stage[group].round(0)
277
278 group_stage['Group D']
279
```

[128]:

|   | Team      | Pts |
|---|-----------|-----|
| 0 | France    | 7.0 |
| 2 | Denmark   | 6.0 |
| 3 | Tunisia   | 3.0 |
| 1 | Australia | 2.0 |

### The Round of 16

According to tournament rules, the top two teams in each group progress into the Round of 16, while the bottom two teams are automatically eliminated.

The fixtures were already predetermined; I just needed to teach the model which team was first and which was second and assign their respective 'names' (group winner & runner-up).

I built another function that would serve all the knockout rounds. The function would predict each game, find the winner and add the winning team's name to a new column called 'winner'.

```

280 df_R_16
281
282 for group in group_stage:
283     group_winner = group_stage[group].loc[0, 'Team']
284     runner_up = group_stage[group].loc[1, 'Team']
285
286     df_R_16.replace({f'Winners {group}': group_winner,
287                     f'Runners-up {group}': runner_up}, inplace=True)
288 df_R_16['winner'] = 'TBD'
289 df_R_16
290
291 def winner(round):
292     for index, row in round.iterrows():
293         home, away = row['home'], row['away']
294         points_home, points_away = prediction(home, away)
295         if points_home > points_away:
296             winner = home
297         else:
298             winner = away
299         round.loc[index, 'winner'] = winner
300     return round
301
302 winner(df_R_16)

```

[136]:

|    | home      | score    | away         | year | winner    |
|----|-----------|----------|--------------|------|-----------|
| 48 | Qatar (H) | Match 49 | Iran         | 2022 | Iran      |
| 49 | Argentina | Match 50 | Australia    | 2022 | Argentina |
| 50 | France    | Match 52 | Saudi Arabia | 2022 | France    |
| 51 | England   | Match 51 | Ecuador      | 2022 | England   |
| 52 | Spain     | Match 53 | Canada       | 2022 | Spain     |
| 53 | Brazil    | Match 54 | Ghana        | 2022 | Brazil    |
| 54 | Belgium   | Match 55 | Costa Rica   | 2022 | Belgium   |
| 55 | Portugal  | Match 56 | Serbia       | 2022 | Portugal  |

## The Quarter-Finals

The next stages followed a repeated pattern, so I built a function that would serve all of them without writing extra lines of code.

The function was designed to extract the winning teams, use the match numbers to place the teams in the next stage as home or away and add a “winner’s” column.

I simply used the function I built earlier to simulate the matches and determine each winner.

```
304 df_qf
305
306 def next_stage(round_1, round_2):
307     for index, row in round_1.iterrows():
308         winner = round_1.loc[index, 'winner']
309         match = round_1.loc[index, 'score']
310         round_2.replace({f'Winners {match}': winner}, inplace=True)
311         round_2['winner'] = 'TBD'
312     return round_2
313
314 next_stage(df_R_16, df_qf)
315
316 winner(df_qf)
```

[144]:

|    | home    | score    | away      | year | winner    |
|----|---------|----------|-----------|------|-----------|
| 56 | Spain   | Match 58 | Brazil    | 2022 | Brazil    |
| 57 | Iran    | Match 57 | Argentina | 2022 | Argentina |
| 58 | Belgium | Match 60 | Portugal  | 2022 | Portugal  |
| 59 | England | Match 59 | France    | 2022 | France    |

### The Final Four

After all of that, Messi (Argentina) and Ronaldo (Portugal) were still within sight of the prize. The next lines of code were the easiest of the entire project to write. It was pretty surreal at this point. Would it actually play out like this? Would there be a Messi vs. Ronaldo final?

```

318 next_stage(df_qf, df_sf)
319
320 winner(df_sf)
321
322 # 3rd and 4th placed teams
323 prediction('Argentina', 'Portugal')
324
325 next_stage(df_sf, df_F)
326
327 winner(df_F)

```

[148]:

|    | home            | score    | away            | year | winner          |
|----|-----------------|----------|-----------------|------|-----------------|
| 60 | Argentina       | Match 61 | Brazil          | 2022 | Brazil          |
| 61 | France          | Match 62 | Portugal        | 2022 | France          |
| 62 | Losers Match 61 | Match 63 | Losers Match 62 | 2022 | Losers Match 62 |

[154]:

|    | home   | score    | away   | year | winner |
|----|--------|----------|--------|------|--------|
| 63 | Brazil | Match 64 | France | 2022 | Brazil |

No.  
Brazil Won.  
(Based on historical data).

### **TL;DR**

Football is played on the pitch by humans, which means predictions can never be accurate. However, there are always favourites and underdogs, and this is based on Data. What I attempted to do here was use historical data to predict a future outcome based on a single question: "Who would win the highly coveted World Cup?"

Brazil won.  
France was the runner-up.  
Argentina took bronze.

Fortunately and unfortunately (however one wishes to look at it), football is played on the pitch, so we shall see if this prediction truly holds water.

Regardless, one thing is certain...I definitely spent too much time on this. *Closes laptop and goes outside to touch grass*

